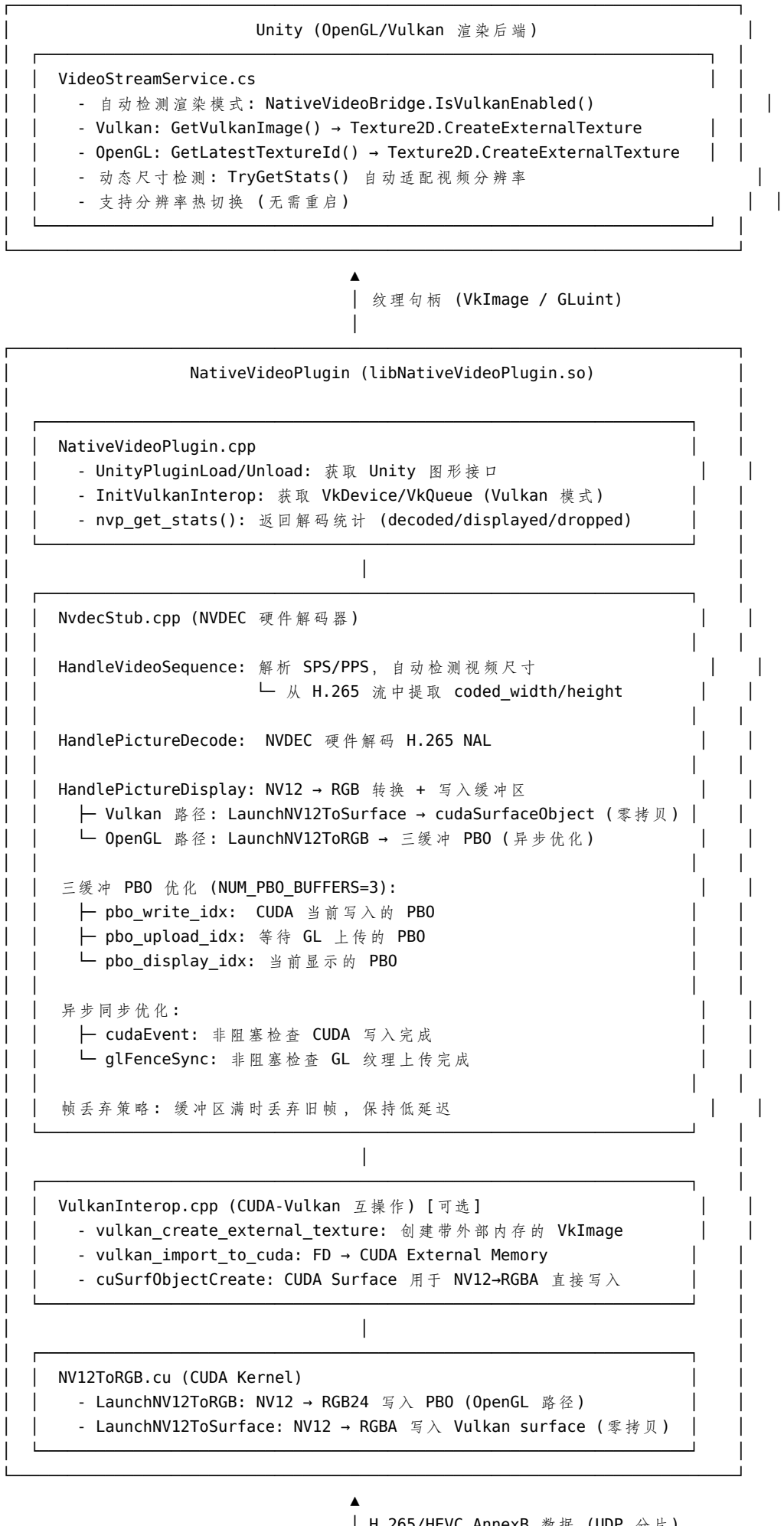


UDP 图传架构说明

最后更新: 2026-01-29

当前状态: **生产就绪** - 支持 2K/4K 120fps, 三缓冲 PBO 优化

1 1. 系统概览



2 2. 图传尺寸控制链路

尺寸自动适配流程

1. MockServer (video_sender.cpp)
 - └─ FFmpeg scale=2560:1440 → 输出 2K H.265 流

↓
2. H.265 流 (UDP 传输)
 - └─ VPS/SPS/PPS NAL 单元包含分辨率信息

↓
3. NVDEC (HandleVideoSequence 回调)
 - └─ ctx->width = pFormat->coded_width (2560)
 - └─ ctx->height = pFormat->coded_height (1440)

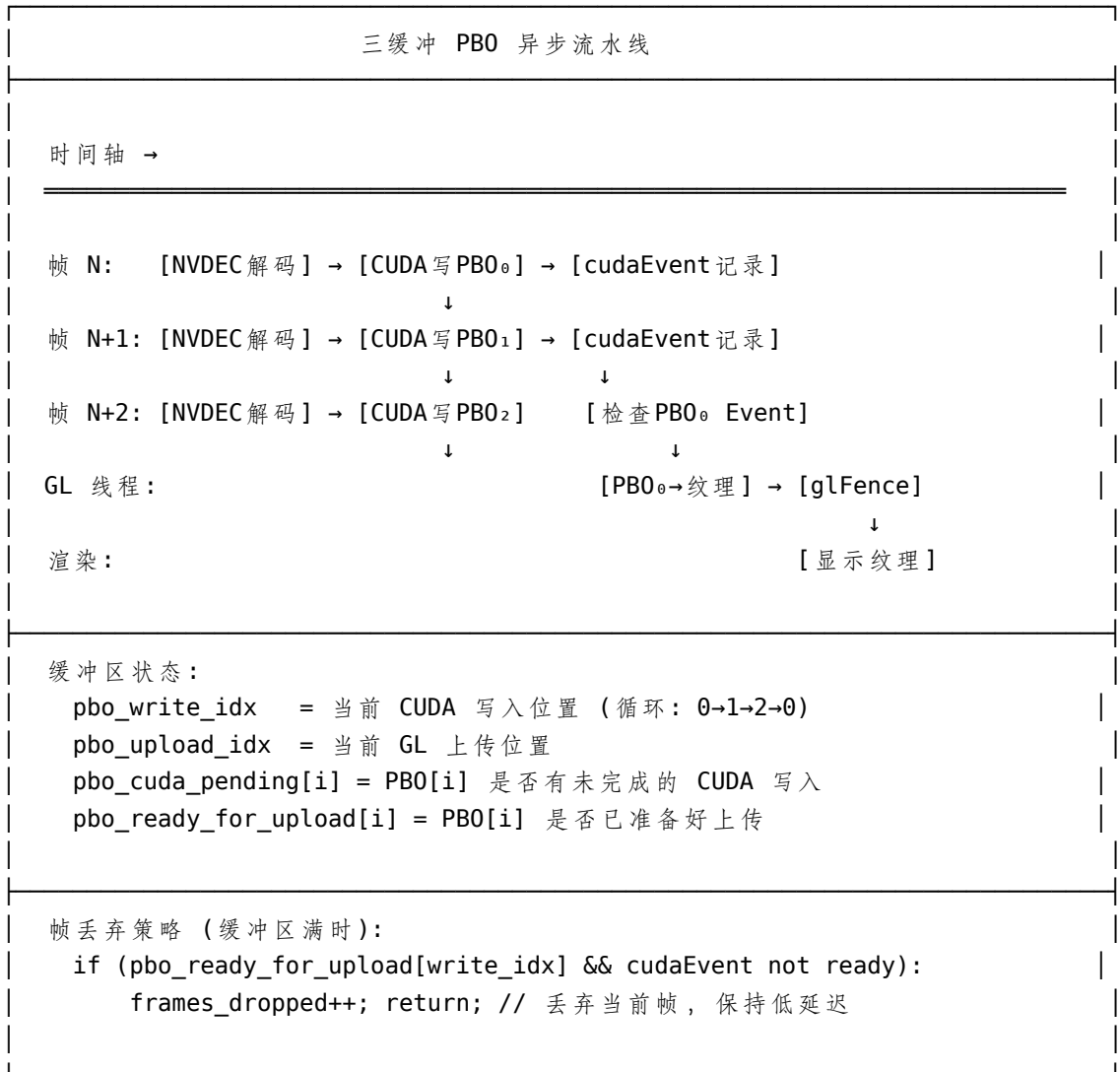
↓
4. PBO 管理 (setup_gl_objects)
 - └─ 预分配 4K 尺寸: MAX_WIDTH=3840, MAX_HEIGHT=2160
 - └─ 实际显示尺寸: gl_width/gl_height = ctx->width/height
 - └─ 三缓冲 PBO: 每个 ~25MB, 总计 ~75MB GPU 内存

↓
5. NativeVideoPlugin (nvp_get_stats)
 - └─ 返回 { width, height, decoded, displayed, dropped }

↓
6. Unity (VideoStreamService.Update)
 - └─ 检测 stats.width != nativeTexWidth?
 - 是: 销毁旧纹理, 创建新尺寸的 Texture2D
 - 否: 继续使用当前纹理

↓
7. RawImage (MainScene.unity)
 - └─ RectTransform Anchor (0,0)-(1,1) 全屏自适应拉伸

3 3. 三缓冲 PBO 工作流程 (OpenGL 路径)



4 4. 数据结构定义 (NvdecStub.h)

```
struct NvdecContext {
    // 4K 120fps 优化常量
    static constexpr int NUM_PBO_BUFFERS = 3;    // 三缓冲 PBO
    static constexpr int MAX_WIDTH = 3840;       // 4K 宽度预分配
    static constexpr int MAX_HEIGHT = 2160;      // 4K 高度预分配
    static constexpr size_t MAX_PBO_SIZE = MAX_WIDTH * MAX_HEIGHT * 3; // ~25MB

    // CUDA 上下文
    CUcontext cuCtx;
    CUvideoctxlock cuLock;
    CUvideodecoder decoder;
    CUvideoparser parser;
    cudaStream_t stream;
```

```
// 视频尺寸 (从流中自动检测)
int width, height;

// OpenGL 三缓冲 PBO
unsigned int pbo[3];           // GL Buffer IDs
unsigned int tex;             // GL Texture ID
CUgraphicsResource cuPbo[3];   // CUDA-GL 互操作资源
int pbo_write_idx;            // CUDA 写入索引
int pbo_upload_idx;           // GL 上传索引
int gl_width, gl_height;      // 当前 GL 对象尺寸

// 异步同步
cudaEvent_t cudaWriteEvent[3]; // CUDA 写入完成事件
bool pbo_cuda_pending[3];       // CUDA 写入进行中
bool pbo_ready_for_upload[3];   // 准备好 GL 上传
void* glFence;                 // GLsync fence

// 统计
std::atomic<int> frames_decoded;
std::atomic<int> frames_displayed;
std::atomic<int> frames_dropped; // 因缓冲区满丢弃的帧

// Vulkan 路径 (可选)
VulkanInteropContext vkCtx;
bool use_vulkan;
cudaSurfaceObject_t vkSurface;
};
```

5 5. 性能指标

指标	2K (2560×1440)	4K (3840×2160)
目标帧率	30-60 fps	30-120 fps
单帧 RGB 大小	~11 MB	~25 MB
三缓冲 PBO 总内存	~33 MB	~75 MB
NVDEC 解码表面	8-12 个	8-12 个
总 GPU 内存占用	~100 MB	~275 MB
延迟 (端到端)	<50ms	<80ms

6 6. 关键文件清单

文件	路径	功能
VideoStreamService.cs	Assets/Scripts/Framework/Video/	Unity 端视频服务, 纹理管理
NativeVideoBridge.cs	Assets/Scripts/Framework/Video/	C# Native 桥接
NvdecStub.h/cpp	Assets/Plugins/NativeVideoPlugin/	NVDEC 解码器 + PBO 管理
NV12ToRGB.cu	Assets/Plugins/NativeVideoPlugin/	CUDA 色彩空间转换内核
VulkanInterop.cpp	Assets/Plugins/NativeVideoPlugin/	Vulkan 互操作 (可选)
video_sender.cpp	Assets/MockServerCpp/src/	MockServer 视频发送
start_mock.sh	Assets/MockServerCpp/	MockServer 启动脚本

7 7. 启动流程

```
# 1. 启动 MockServer (视频源)
cd Assets/MockServerCpp
./start_mock.sh                # 自动选择 Test_4k_120fps.mp4

# 2. 启动 Unity (OpenGL 模式)
./RoboMasterClientWMJ -force-opengl

# 3. 监控日志
tail -f Log/DebugLog.txt | grep -E "NVDEC|VideoStream"
```

8 8. 故障排查

问题	可能原因	解决方案
黑屏无图像	MockServer 未启动	<code>./start_mock.sh</code>
画面撕裂	双缓冲不足	已升级为三缓冲
帧率低	GPU 内存不足	检查 <code>nvidia-smi</code>
尺寸错误	PBO 未重建	会自动检测并重建
CUDA-GL 失败	GPU 不匹配	确保 Unity 使用 NVIDIA GPU

9 9. 版本历史

- **v1.0:** 初始双缓冲 PBO 实现
- **v1.1:** 添加 Vulkan 路径支持
- **v1.2:** 修复视频尺寸检测
- **v2.0:** 三缓冲 PBO + 异步 CUDA Event + GL Fence 优化
- **v2.1:** 4K 预分配 + 帧丢弃策略 (当前版本)